

✓ Assignment Supervised machine Learning - Regression

House Price Prediction Using Bolton Housing Data

This dataset provides information about house prices in Bolton. House Price Prediction

In this notebook, you'll follow the basic machine learning process to build a regression model to predict house prices using the "Boston Housing Dataset" from sklearn. The regression model will either be a Decision Tree or Random Forest regressor.

Follow the instructions and complete each TODO to complete the assessment on the essential steps in building and evaluating a regression model.

The following is a description of each column in the dataset:

Dataset Features (Bolton Housing):

- CRIM: Crime rate by town
- ZN: Proportion of residential land zoned for large lots
- INDUS: Proportion of non-retail business acres per town
- CHAS: Charles River dummy variable (1 if tract bounds river; 0 otherwise)
- NOX: Nitric oxide concentration (parts per 10 million)
- RM: Average number of rooms per dwelling
- AGE: Proportion of owner-occupied units built before 1940
- DIS: Weighted distances to five Boston employment centers
- RAD: Index of accessibility to radial highways
- TAX: Full-value property tax rate per \$10,000
- PTRATIO: Pupil-teacher ratio by town
- B: Proportion of Black population
- LSTAT: Percentage of lower status of the population
- MEDV (Target): Median value of owner-occupied homes in \$1,000s

Dataset is from sklearn Datasets

```
# --- Imports ---  
# TODO: Import all the necessary  
# Example: import pandas as pd  
# Import required libraries
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning Libraries
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV
```

```
# --- Data Collection and Loading ---
# TODO: Load the 'Boston Housing' dataset from sklearn and convert it into a pandas DataFrame
# Hint: Use `load_boston()` from `sklearn.datasets`

# Load dataset and convert to DataFrame:

df = pd.read_csv("housing_Data.csv")

# Display first five rows
df.head()
```

ad	guestroom	basement	h
'es	no	no	
'es	no	no	
'es	no	yes	
'es	no	yes	
'es	yes	yes	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
# --- Quick Check of Data ---
# TODO: Display the first few rows of the dataset to understand its structure.
# Hint: Use `.head()` to inspect the first few rows.

# Display first five rows
print(df.head().to_string())
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotw
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

```
# TODO: Check the features and target variable. Identify which is continuous ar
# Hint: Use `.info()` and `.describe()` to inspect data types and statistical p
print("To inspect data types")
print(df.info())
```

```
print("\n", "statistical properties")
print(df.describe().round(2).to_string())
```

To inspect data types

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	price	545 non-null	int64
1	area	545 non-null	int64
2	bedrooms	545 non-null	int64
3	bathrooms	545 non-null	int64
4	stories	545 non-null	int64
5	mainroad	545 non-null	object
6	guestroom	545 non-null	object
7	basement	545 non-null	object
8	hotwaterheating	545 non-null	object
9	airconditioning	545 non-null	object
10	parking	545 non-null	int64
11	prefarea	545 non-null	object
12	furnishingstatus	545 non-null	object

```
dtypes: int64(6), object(7)
```

```
memory usage: 55.5+ KB
```

```
None
```

statistical properties

	price	area	bedrooms	bathrooms	stories	parking
count	545.00	545.00	545.00	545.00	545.00	545.00
mean	4766729.25	5150.54	2.97	1.29	1.81	0.69
std	1870439.62	2170.14	0.74	0.50	0.87	0.86
min	1750000.00	1650.00	1.00	1.00	1.00	0.00
25%	3430000.00	3600.00	2.00	1.00	1.00	0.00
50%	4340000.00	4600.00	3.00	1.00	2.00	0.00
75%	5740000.00	6360.00				
max	13300000.00	16200.00				



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```
# --- EDA and Data Preprocessing ---
# TODO: Check for missing/null values.
# Hint: Use `.isnull().sum()` to check for null values.

print("To check for null values",df.isnull().sum())

df = pd.get_dummies(df, drop_first=True)
print("Converted the categorical variables:",df.head().to_string())
```

```
To check for null values price          0
area          0
bedrooms      0
bathrooms     0
stories       0
parking       0
mainroad_yes  0
guestroom_yes 0
basement_yes  0
hotwaterheating_yes 0
airconditioning_yes 0
prefarea_yes  0
furnishingstatus_semi-furnished 0
furnishingstatus_unfurnished 0
dtype: int64
Converted the categorical variables:      price  area  bedrooms  bathrooms  sto
0  13300000  7420      4      2      3      2      True
1  12250000  8960      4      4      4      3      True
2  12250000  9960      3      2      2      2      True
3  12215000  7500      4      2      2      3      True
4  11410000  7420      4      1      2      2      True
```

```
# TODO: Visualize the data. Create scatter plots to see the relationship between
# Example: Use `plt.scatter()` to visualize the relationship between features 1

# List of numerical features
features = ['area', 'bedrooms', 'bathrooms', 'stories', 'parking']

# TODO: Create a function to automate scatter plots for all features vs the tar
# Hint: The function should loop over a list of features and plot scatter plots

# Define a function to create scatter plots
def scatter_plots(data, feature_list, target):
    for feature in feature_list:
        plt.figure(figsize=(6,4))
        plt.scatter(data[feature], data[target])
        plt.xlabel(feature)
        plt.ylabel(target)
        plt.title(f'{feature} v
        plt.show()
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```
# TODO: Use the function to visualize the relationships between multiple featur  
scatter_plots(df, features, 'price')
```



McAfee | WebAdvisor



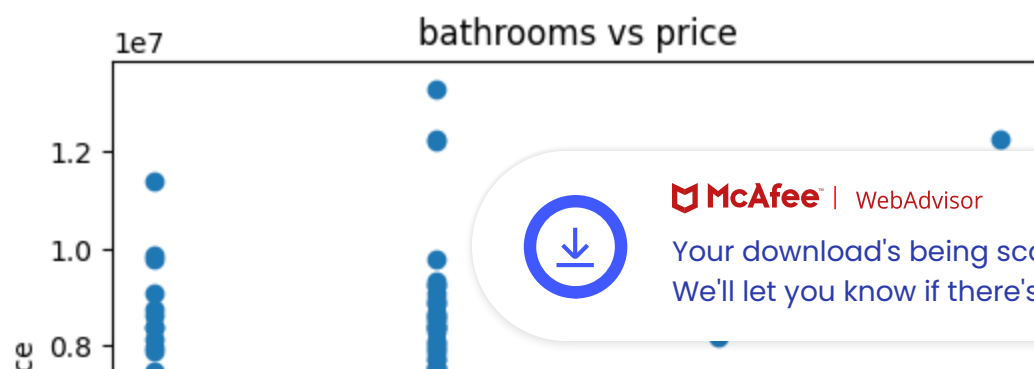
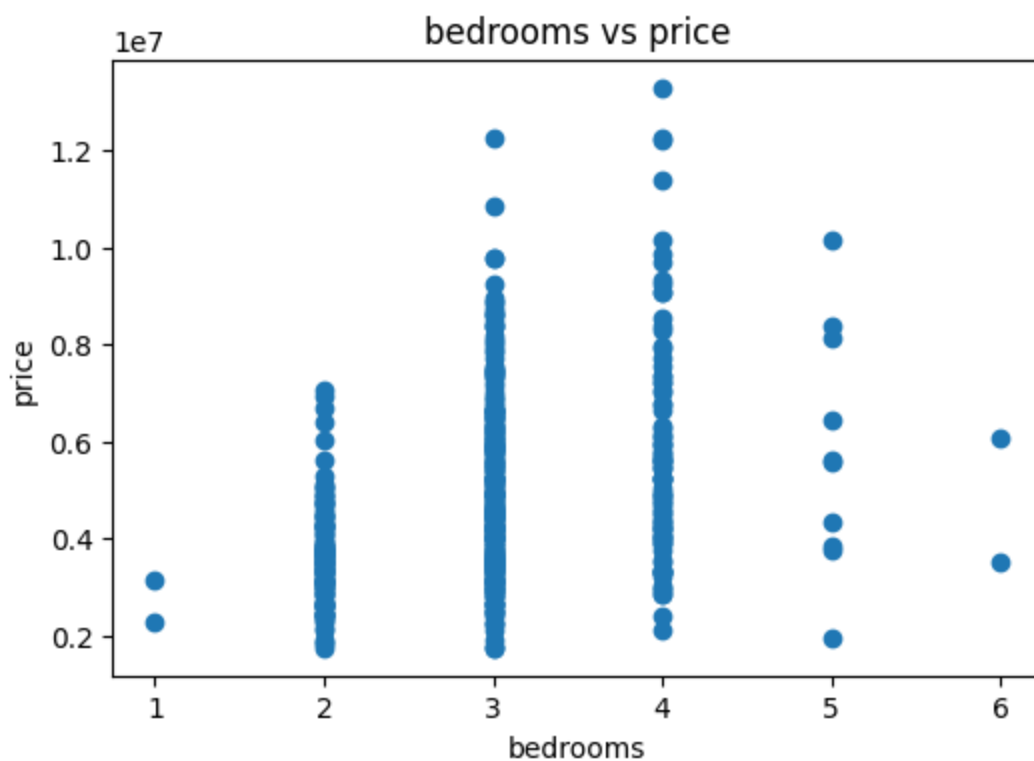
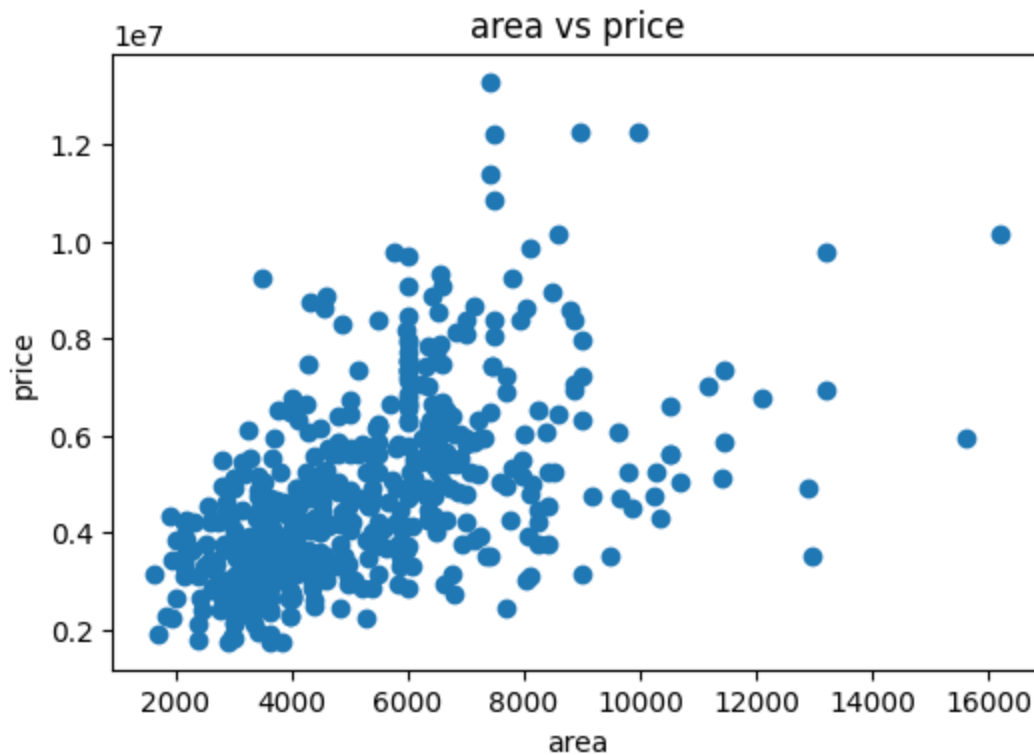
Your download's being scanned.
We'll let you know if there's an issue.



McAfee | WebAdvisor



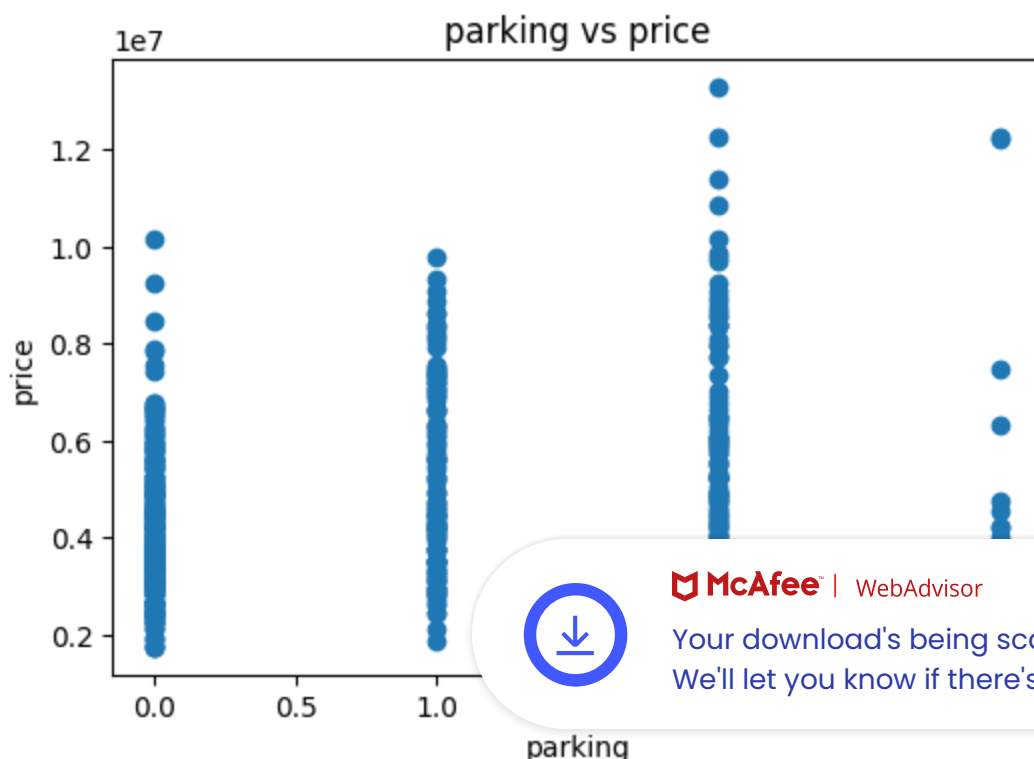
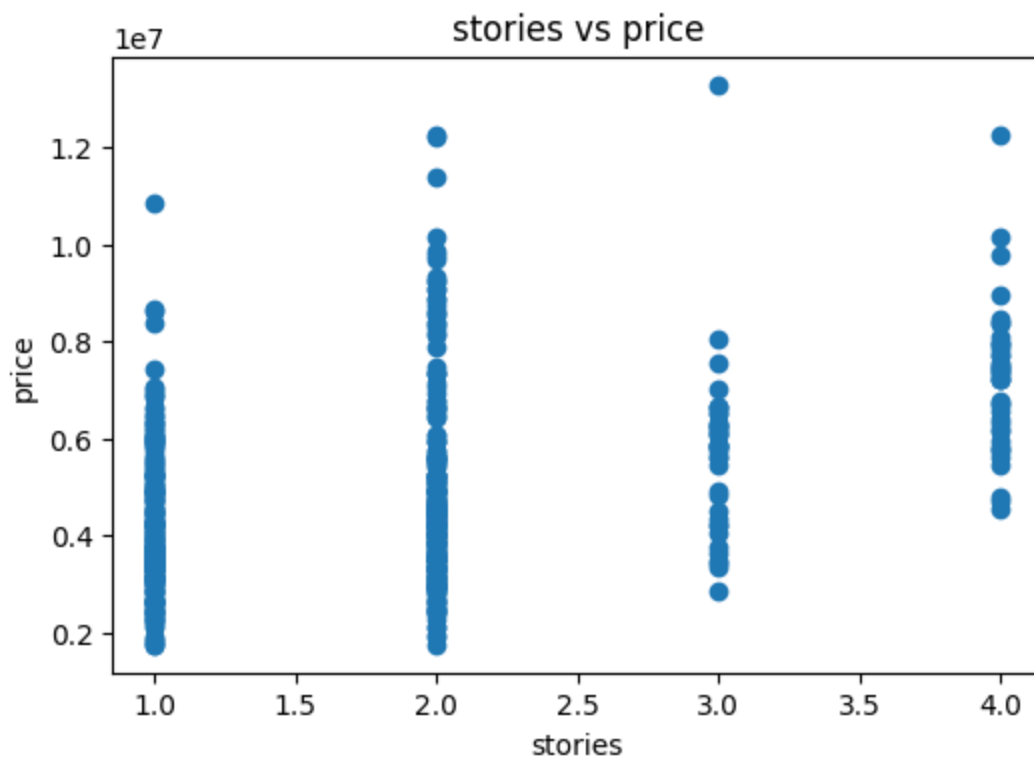
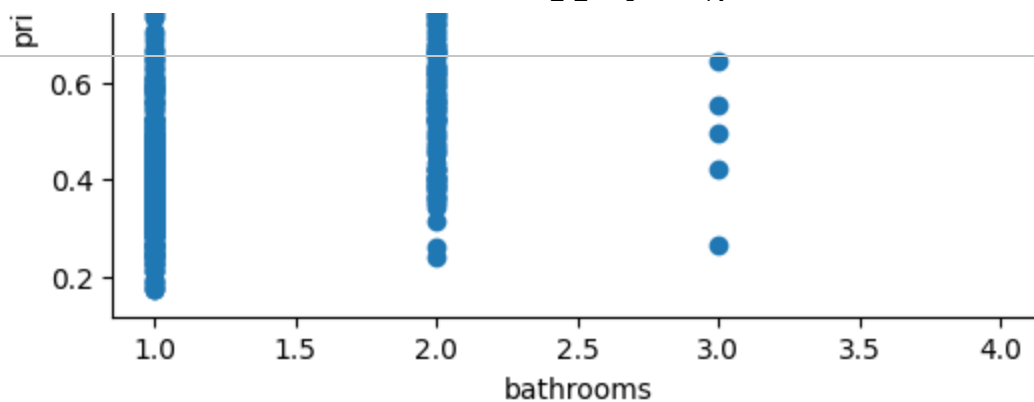
Your download's being scanned.
We'll let you know if there's an issue.



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

**McAfee** | WebAdvisor

Your download's being scanned.
We'll let you know if there's an issue.

Observation: The scatter plots show the relationship between the selected numerical features and house price. Features such as area, bathrooms, and parking have a stronger positive relationship with the target variable (price). The function makes it easy to generate multiple scatter plots without writing the same code repeatedly.

```
# --- ML Model Training ---
# TODO: Split the dataset into training and testing sets.
# Hint: Use `train_test_split()` from `sklearn.model_selection` with an 80/20 s

# Define X (features) and y (target) and perform the train-test split:
X = df.drop('price', axis=1) # All columns except 'price'
y = df['price']              # Target variable

# Split the dataset into training and testing sets (80% training, 20% testing)

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_st

# Display the shapes of the datasets
print("Training Features:", X_train.shape)
print("Testing Features:", X_test.shape)
print("Training Target:", y_train.shape)
print("Testing Target:", y_test.shape)
```

```
Training Features: (436, 13)
Testing Features: (109, 13)
Training Target: (436,)
Testing Target: (109,)
```

Observation: The dataset was split into 80% training data and 20% testing data. The training set has 436 records, and the testing set has 109 records. Both datasets contain 13 features. The training data is used to train the model, and the testing data is used to check how accurately the model predicts house prices.

```
# TODO: Choose an appropriate regression model: Decision Tree or Random Forest.
# Hint: Use either `DecisionTreeRegressor` or `RandomForestRegressor` from `skl

# Define your regression model here:

randfor_model = RandomForestRegressor( n_estimators=100,random_state=42)
```

Observation: A Random Forest Regressor uses many decision trees to improve prediction accuracy.



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.